SCRIBBLE TO DIGITAL

Team ID : SWTID-2026-3789

Team size : 5

Team Leader : SRI PADMA PRIYA V

Team member : SUBASHREE K

Team member : DHARSHANA N

Team member : PAVITHRA M

Team member : PAVITHRA S

PROJECT DESCRIPTION

Converting handwritten notes into digital text is a common challenge for students and professionals, especially when poor lighting, messy handwriting, and mixed content make notes difficult to read and organize.

Scribble to Digital is an AI-powered system designed to solve this problem by converting handwritten notes into clean and structured digital text. Users can upload an image of their notes captured through a mobile phone or scanner, and the system enhances the image to improve clarity and remove noise.

It then uses Optical Character Recognition (OCR) to extract the text, while Generative AI corrects grammar and improves readability. The system also identifies important To-Do tasks from the notes and organizes them into a structured task list. Developed using Streamlit for the frontend and Python for the backend, the application combines image

processing, OCR, and AI technologies to help users easily transform handwritten notes into organized digital content and improve productivity.

USE CASE SCENARIOS

Scenario 1: Handwritten Note Digitization

Users upload photos of their handwritten notes using a mobile phone or scanner. The system enhances the image by improving brightness, contrast, and removing noise. Then, OCR (Optical Character Recognition) extracts the text.

Output: Clean, editable digital text that preserves the content of the original notes.

Scenario 2: Intelligent Text Correction

After OCR extraction, the text may contain spelling mistakes or unclear words. The Generative AI analyzes the text to correct grammar, fix spelling errors, reconstruct broken words, and format it into readable paragraphs.

Output: Professional, structured, and easy-to-read digital notes.

Scenario 3: Automatic To-Do Task Extraction

Handwritten notes often include tasks mixed with general information. The AI identifies task-related sentences, extracts actionable items, and organizes them into a clear To-Do list.

Output: A structured list of tasks separated from regular notes, improving task management.

Scenario 4: Digitizing Sketches and Diagrams

Users can upload rough sketches, diagrams, or flowcharts. The system detects shapes, lines, and connections to create clean digital diagrams that are editable.

Output: Neat and organized diagrams that maintain the structure of the original sketch.

Scenario 5: Multilingual Note Recognition

The system supports notes written in different languages or mixed scripts. OCR and AI detect the language, extract the text, and correct grammar based on the language rules.

Output: Digital notes in the correct language, ready for use or sharing.

ARCHITECTURE OVERVIEW

Scribble to Digital is an AI-powered web application designed to convert handwritten notes into clean, structured digital text. The platform is built using the Streamlit framework, providing an interactive and user-friendly interface for uploading images and viewing results.

The system integrates:

- Streamlit for frontend interface and application logic

- OpenCV for image preprocessing and enhancement
- OCR Engine (Tesseract/EasyOCR) for text extraction
- Generative AI (Google Gemini 2.5 Flash) for contextual correction, formatting, and task extraction
- Python as the core backend programming language
- Optional modules like ReportLab and Python-DOCX for exporting to PDF or Word

The platform workflow involves: uploading handwritten notes, enhancing image clarity, extracting text, correcting grammar and formatting with AI, and generating structured outputs such as digital notes and To-Do lists.

Core Technologies

- Streamlit – Builds the interactive web interface for uploading images, displaying extracted text, corrected content, and task lists.
- Google Gemini 2.5 Flash – AI model for contextual understanding, grammar correction, and automatic task extraction.
- OpenCV – Image preprocessing including grayscale conversion, noise removal, contrast/brightness adjustment, and thresholding for better OCR accuracy.
- OCR (Tesseract/EasyOCR) – Converts enhanced handwritten images into machine-readable text.

- Python – Main programming language for backend processing and AI integration.
- ReportLab / Python-DOCX – Optional modules for exporting processed content to PDF or Word formats.

Pre-requisites

- Google AI Studio Account – To generate Gemini API Key
- Python 3.8+ installed
- Streamlit Framework installed

```
pip install streamlit
```

- Required python libraries:
  - opencv-python
  - pytesseract / easyocr
  - google-generativeai
  - pillow
  - reportlab
  - python-docx

PROJECT WORKFLOW

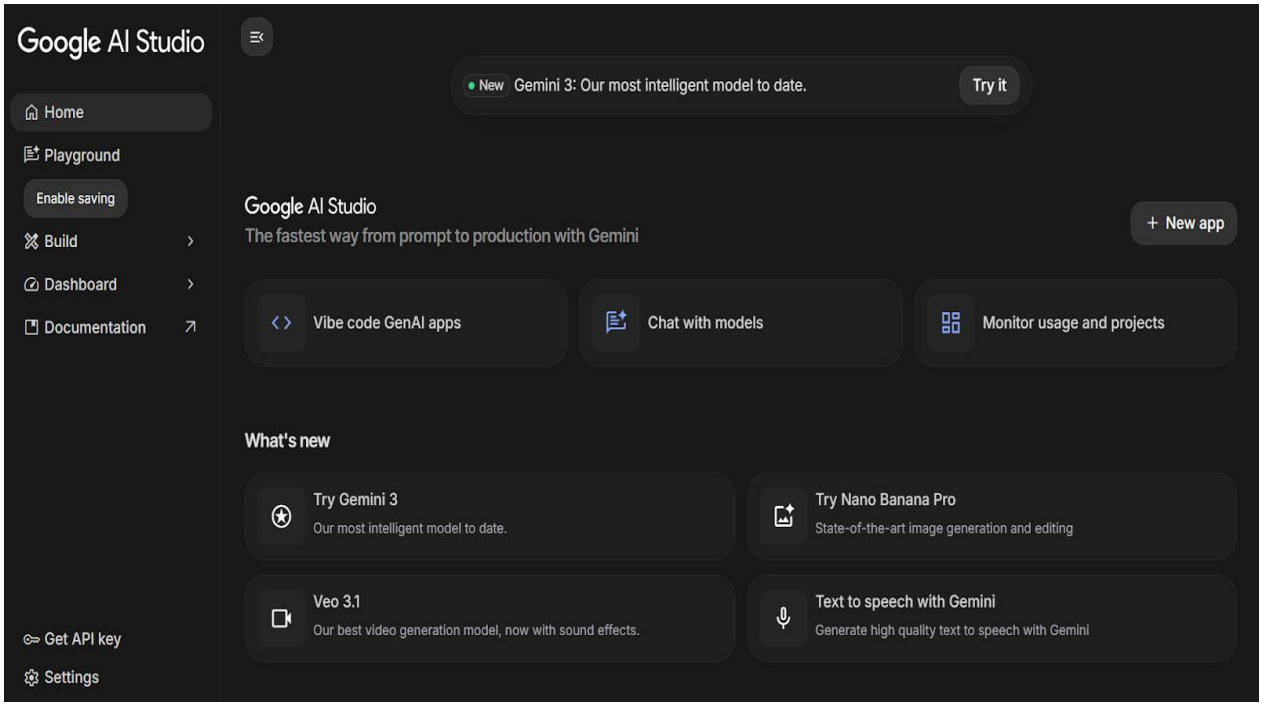
Scribble to Digital is an AI-powered web application that converts handwritten notes into structured digital text. The workflow is divided into five milestones, covering AI setup, core functionality development, UI design, and testing.

MILESTONE 1: Generative AI Setup & Initialization

Objective: Configure AI infrastructure for handwritten note processing using Google Gemini 2.5 Flash.

Activity 1.1: Google AI Studio Setup

- Sign in or create a Google AI Studio account.
- Generate the Gemini API key and enable Gemini 2.5 Flash.
- Test API connectivity using sample prompts.



Activity 1.2: Configure Environment

- Store the API key in a .env file.
- Load environment variables using python-dotenv.

- Configure model parameters: temperature, max output tokens, and response format.

```
scribble_to_digital/
    app.py
    .env
```

Add:

```
GEMINI_API_KEY=your_api_key_here
```

Activity 1.3: Design AI Prompts

- Create prompts for grammar correction, sentence reconstruction, and structured note formatting.
- Design separate prompts for task extraction and prioritization.
- Test prompt effectiveness with sample handwritten text.

Input:

```
i am go school tomorrow
```

Output:

```
I am going to school tomorrow.
```

MILESTONE 2: Core Functionality Development

Objective: Build backend modules and implement OCR and AI features.

Activity 2.1: Streamlit App Structure

- Create a modular project structure.
- Configure layout: title, description, file uploader, processing buttons, and output display sections.

File Explorer Structure

```
scribble_to_digital/
|
├── app.py                  # Main Streamlit application
├── .env                    # API key storage
├── requirements.txt        # Python dependencies
|
├── utils/
|   ├── ocr_processor.py    # Image preprocessing & OCR
|   ├── ai_processor.py     # Gemini integration
|   └── file_export.py      # Export functions (PDF / TXT)
|
├── assets/
|   └── sample_images/
|
└── outputs/                # Generated files
```

Activity 2.2: Image Processing Module

- Accept uploaded images (JPG/PNG).
- Enhance images using OpenCV:
    - Grayscale conversion
    - Brightness and contrast adjustment
    - Noise removal
    - Thresholding

- Display enhanced image preview.

OCR Processing Module:

```
import cv2
import pytesseract
import numpy as np

def extract_text_from_image(uploaded_image):
    image = np.array(uploaded_image)
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gray = cv2.threshold(gray, 150, 255, cv2.THRESH_BINARY)[1]

    text = pytesseract.image_to_string(gray)
    return text
```

Activity 2.3: OCR Extraction Module

- Integrate Tesseract or EasyOCR to extract text from enhanced images.
- Clean unnecessary symbols and display raw OCR output.

Activity 2.4: AI-Based Contextual Processing

- Send OCR text to Gemini AI for grammar correction, sentence reconstruction, and formatting.
- Extract actionable tasks into a structured To-Do list.

Create utils/ai_processor.py

```python
import google.generativeai as genai

model = genai.GenerativeModel("models/gemini-2.5-flash")

def clean_and_structure_text(raw_text):
    prompt = f"""
    You are an AI note cleaner.
    1. Correct spelling and grammar.
    2. Reconstruct incomplete words.
    3. Format text properly.
    4. Extract action items as bullet points.

    Raw Text:
    {raw_text}
    """

    response = model.generate_content(prompt)
    return response.text
```

MILESTONE 3: Streamlit UI Implementation

Objective: Build an interactive and user-friendly frontend.

Activity 3.1: UI Components

- st.file_uploader() for image uploads.
- st.image() to display uploaded and enhanced images.
- st.button() for processing actions and st.spinner() for loading feedback.
- st.text_area() to display extracted and corrected text.

Activity 3.2: Display Structured Results

- Sections for: enhanced image, raw OCR text, AI-corrected notes, and extracted tasks.
- Optional checkbox to show/hide raw OCR text.

Activity 3.3: Export Functionality

- Enable downloads in PDF (ReportLab), Word (.docx), and CSV (task list).
- Allow preview before download and secure temporary file cleanup.

Create utils/file_export.py

Save as TXT

```
def save_as_txt(content, filename):
    with open(f"outputs/{filename}.txt", "w", encoding="utf-8") as f:
        f.write(content)
```

In app.py:

```
if st.button("Download as TXT"):
    save_as_txt(clean_text, "digital_notes")
    st.success("File saved in outputs folder.")
```

MILESTONE 4: Testing & Optimization

Objective: Ensure accuracy, reliability, and performance.

Activity 4.1: Functional Testing

- Test with clear, medium, and messy handwriting.
- Validate OCR accuracy and AI corrections.
- Verify To-Do task extraction.

Activity 4.2: AI Response Quality

- Check grammar correction, logical sentence flow, and structured formatting.
- Ensure extracted tasks are correct and complete.

Activity 4.3: Performance Optimization

- Optimize image preprocessing for faster processing.
- Improve AI prompt clarity and reduce token usage.
- Handle empty images, OCR failures, and API errors gracefully.

Activity 4.4: Export Quality Testing

- Validate PDF, Word, and CSV formatting.
- Test downloads across different browsers.

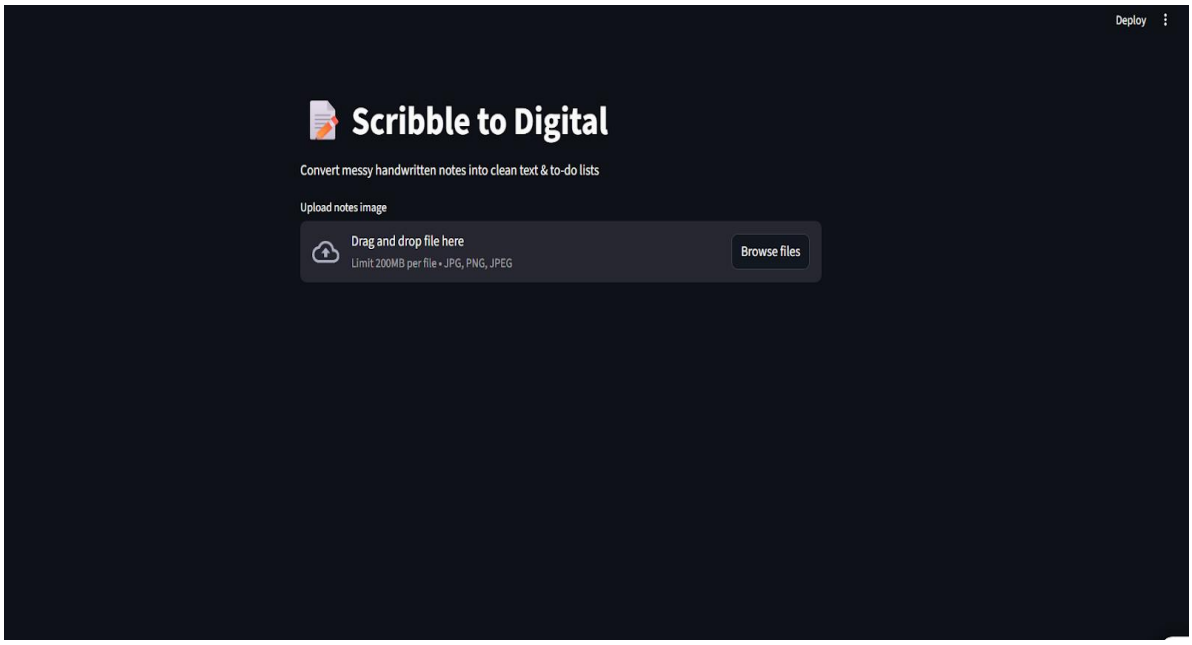
MILESTONE 5: Complete User Interaction Flow

Objective: Provide a smooth and logical workflow for users.

- User opens the Streamlit app and views the landing page.

- Uploads handwritten notes (JPG/PNG).
- Clicks Convert to process notes via OCR and AI.
- Views: enhanced image, raw OCR text, AI-corrected digital notes, and extracted tasks.
- Downloads structured notes in PDF, Word, or CSV formats.

Landing Page Development:



CONCLUSION

Scribble to Digital converts traditional handwritten notes into clean, structured digital text using AI.The system combines OCR technology with Generative AI for accurate text extraction and grammar correction. Image preprocessing ensures that even messy or low-quality notes are easily readable.

Generative AI formats the text into clear paragraphs and extracts important To-Do tasks.The Streamlit frontend provides a simple, interactive, and user-friendly interface.Each module focuses on accuracy, efficiency, and reliability.The application organizes handwritten content into digital notes and structured task lists.

Users save time and effort by avoiding manual rewriting.With optimized performance and error handling, the system remains stable and responsive.Scribble to Digital is a smart tool for students, professionals, and teams to enhance productivity.